Primality Testing

CS648 Course Project Report

by Prerak Agarwal, Suryansh Goel and Raghav Madan

Under the supervision of Dr. Surender Baswana

April 2025

Contents

1 Acknowledgement

We express our gratitude to **Professor Surender Baswana** for his unwavering guidance and support throughout the duration of this project. His assistance significantly influenced the course of our journey, making it markedly distinct from what it would have been otherwise.
We approached **Professor Surender Baswana** with a lot of questions in mind about what all should we include in our presentation and how to present this effort of months in just 10 minutes.He replied with a smile that "It should be a **perfect** presentation of 10 minutes". His words really hit us and therefore our presentation and report is made with the objective to achieve **perfection**.

Group Members

Name	**Roll Number**	**Email ID**
Suryansh Goel	221111	suryanshg22@iitk.ac.in
Prerak Agarwal	220818	prerakag22@iitk.ac.in
Raghav Madan	220853	raghavm22@iitk.ac.in

2 Introduction

Primality testing is a fundamental problem in number theory and computer science that involves determining whether a given number is prime. Primality testing plays a crucial role in areas such as cryptography (e.g., RSA encryption), random number generation, and computational mathematics.

Primality testing has vital applications in cryptography, especially in generating large prime numbers used in public-key algorithms like RSA, Diffie-Hellman, and Elliptic Curve Cryptography (ECC). It ensures the security of encrypted communication by enabling the creation of cryptographic keys that are difficult to factor. Beyond cryptography, primality testing is used in random number generation, digital signatures, and blockchain protocols.

Prof. Manindra Agrawal, Prof. Neeraj Kayal, and Prof. Nitin Saxena, in 2002, showed whether a number is prime or not, in polynomial time in size (approximately $O(m^{12})$ time where m represents the number of bits in a number n)[1]. However, the implementation is non-trivial and involves complex computations.

In this project we rediscovered an $O(k.m^2)$ (where m represents the number of bits in a number n) randomized Monte Carlo algorithm which is based on properties of modular arithmetic and builds on **Fermat's Little Theorem**. Our algorithm uses elementary number theoretic results and is easy to implement.

We verified our algorithm by running it over various inputs of different sizes and comparing it with naive deterministic algorithms.

3 Naive Algorithms

In order to check whether a number is prime or not, the most basic approach is to find an improper divisor of n(if it exists). Since every divisor is less than equal to that number, the naive approach requires iterating from 1 to n(excluding 1 and n) and checking whether it divides n. The worst time complexity of this algorithm is $O(n)$.

Another approach(still naive) is to iterate from 1 to $\sqrt{n}$, since, if d divides n, then $k = n/d$ also divides n. Another optimization in this algorithm can be to consider only odd numbers from 1 to $\sqrt{n}$, since 2 is the only even prime

number. The worst time complexity of these algorithms is $O(\sqrt{n})$.

Another approach(generally used for smaller primes) is **Sieve of Eratosthenes**, which returns a boolean array of size n, indicating whether $x \in \{1, 2, 3...n-1\}$ is prime or not. This algorithm works on the principle that if d is prime, then all the multiples of d are composite(excluding d). The algorithm requires iterating x from 1 to $\sqrt{n}$ and marking all the multiples of x as composite, if x is prime. The time complexity of this algorithm is $O(nloglogn)$ and requires $O(n)$ space.

These approaches, although deterministic, take much more computation time for large numbers.

4 Incremental Approach

We will now try to build the primality testing algorithm incrementally. Before that, we will state **Fermat's Little Theorem**.

4.1 Fermat's Little Theorem

The number n, is prime if and only if $a^{n-1} = 1(mod\, n)$ for every integer $a \in \{1, 2, 3, .., n-1\}$.

Proof: Consider the set of integers:

$$S = \{a, 2a, 3a, \ldots, (p-1)a\}$$

We take all these values modulo p. Since $\gcd(a, p) = 1$, each of the elements in S is distinct modulo p. That is, for $1 \leq i < j \leq p-1$, if

$$ia \equiv ja \pmod{p} \Rightarrow (i-j)a \equiv 0 \pmod{p}$$

Since $p \nmid a$, we must have $i \equiv j \pmod{p}$, implying $i = j$.

Thus, the set $\{a, 2a, \ldots, (p-1)a\} \mod p$ is a rearrangement of the set $\{1, 2, \ldots, p-1\} \mod p$.

Multiplying all elements in both sets, we get:

$$a \cdot 2a \cdot 3a \cdots (p-1)a = a^{p-1} \cdot (p-1)!$$

and

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod{p}$$

Since $p \nmid (p-1)!$, we can cancel $(p-1)!$ from both sides:

$$a^{p-1} \equiv 1 \pmod{p}$$

Hence proved.

□

4.2 Terminology

We will introduce some definitions in this section for **composite numbers**, which will help us to state the algorithm later.

Fermat Witnesses: Consider

$$T = \{a : a < n \, and \, a^{n-1} \neq 1(mod n)\}$$

The elements of T are called **Fermat witnesses** to the fact that n is composite.

Divisor Witnesses: Consider

$$D = \{a : 1 < a < n \, and \, a|n\}$$

The elements of D are called **divisor witnesses** to the fact that n is composite.

GCD Witnesses: Consider

$$G = \{a : a < n \, and \, gcd(a, n) > 1\}$$

The elements of G are called **GCD witnesses** to the fact that n is composite.

The following theorems provide a key insight which led us to the incremental algorithm.

4.3 Relation among witnesses

Theorem 1. *Every divisor witness is also a Fermat witness.*

Proof. Suppose that there exists a divisor witness d such that $d \notin T$(T is the set of Fermat witnesses as defined above).

This implies,

$d^{n-1} = 1(mod \, n)$

$\implies$ $d^{n-1} = qn + 1$, for some natural number q

$\implies$ $d^{n-1} - qn = 1$

Since d divides n, thus d divides the expression on the left hand side, However,

d does not divide the expression on the right hand side, which contradicts the fact that $d \notin T$.

Thus, $d \in T$. Hence, proved.

□

Theorem 2. *Every GCD witness is also a Fermat witness.*

Proof. Suppose that there exists a GCD witness d such that $d \notin T$(T is the set of Fermat witnesses as defined above.

Since d is a GCD witness,

Let $gcd(d, n) = k(k \neq 1)$, thus $d = kp$ and $n = kq$, for some natural numbers p and q .

Since d is not a Fermat witness,

This implies,

$d^{n-1} = 1(mod\, n)$

$\implies d^{n-1} = rn + 1$, for some natural number r

$\implies (kp)^{n-1} - rkq = 1$

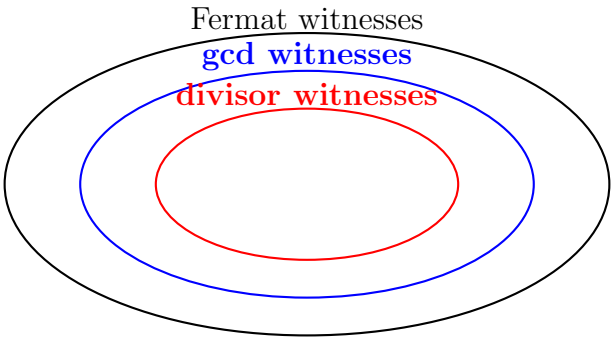
Note that k divides the expression on the left hand side, However, k does not divide the expression on the right hand side, which contradicts the fact that $d \notin T$. Thus, $d \in T$. Hence, proved.

□

Theorem 3. *Every divisor witness is also a GCD witness.*

Proof. If d is a divisor witness, then $d|n$, which implies $gcd(d, n) = d > 1$, then d is also a GCD witness.

However, the converse is not true. Consider $n = 21$ and $d = 15$, d does not divide n, but $gcd(d, n) = 3 > 1$, thus d is a GCD witness, but not a divisor witness. □



In the next section, we will compare the order of number of each type of witness.

4.4 Bounds on Types of Witnesses in Primality Testing

Let n be a composite number, and define $\phi(n)$ as Euler's totient function [3], i.e., the number of integers $a < n$ such that $\gcd(a, n) = 1$. Note that order of Euler's totient function is $O(n)$ and $\Omega(\frac{n}{loglogn})$ for large n.

4.4.1 GCD Witnesses

Since these are the integers $a < n$ for which $\gcd(a, n) > 1$. The number of such values is equal to:

$$\boxed{n - \phi(n)}$$

These are the values not coprime with n, and hence can reveal compositeness directly by computing $\gcd(a, n)$.

4.4.2 Fermat Witnesses

Since every GCD witness is a Fermat witness, thus number of Fermat witnesses is at least:

$$\boxed{|T| \geq n - \phi(n)}$$

To find a better lower bound, we will state a theorem. But we need to classify Fermat witnesses into two categories.

We know that, by theorem 2, that every GCD witness is also a Fermat witness, thus we will call GCD witnesses as **trivial Fermat witnesses**, and those Fermat witnesses which are not GCD witnesses as **non-trivial Fermat witnesses**.

Theorem 4. *For any composite number n, let S denote the set of elements $\leq n$ such that $gcd(a, n) = 1$. Suppose that there is at least one non-trivial Fermat witness for n. Then at least half the elements of S are Fermat witnesses for n.*

Proof. Let

$$P = \left\{a \mid a < n,\ \gcd(a, n) = 1,\ a^{n-1} \equiv 1 \pmod{n}\right\}$$

$$S = \{a \mid a < n,\ \gcd(a, n) = 1\}$$

$$P' = S \setminus P$$

Let $x \in P' \Rightarrow x^{n-1} \not\equiv 1 \pmod{n}$

Consider:

$$B = \{(a \cdot x) \bmod n \mid a \in P\}$$

We aim to prove:

1. $B \subseteq P'$
2. $|B| = |P|$

1) Prove that $B \subseteq P'$:

$$\begin{aligned}[(ax) \bmod n]^{n-1} &= \left(a^{n-1} \bmod n\right) \cdot \left(x^{n-1} \bmod n\right) \\ &= 1 \cdot x^{n-1} \bmod n \\ &\not\equiv 1 \pmod{n} \quad (\text{since } x \in P') \\ \Rightarrow (ax) \bmod n &\in P'\end{aligned}$$

Hence, $B \subseteq P'$.

2) Prove that $|B| = |P|$:
Suppose:

$$(a_1 x) \equiv (a_2 x) \pmod{n} \Rightarrow (a_1 - a_2)x \equiv 0 \pmod{n}$$

Since $\gcd(x, n) = 1$, we get:

$$a_1 \equiv a_2 \pmod{n} \Rightarrow a_1 = a_2 \quad (\text{within the range})$$

So the mapping $a \mapsto ax \bmod n$ is injective on P, implying:

$$|B| = |P|$$

Thus, there exists an injection from $P \to P' \Rightarrow |P| \leq |P'|$.

Since $|P| + |P'| = |S|$, we get:

$$|S| \leq |P| + |P'| \leq 2|P'| \Rightarrow \boxed{\frac{|S|}{2} \leq |P'|}$$

Hence proved. □

Thus, by the above theorem, if n has at least one non-trivial Fermat witness, $|T| \geq (n - \phi(n)) + \frac{\phi(n)}{2}$
$\Longrightarrow$

$$\boxed{|T| \geq n - \frac{\phi(n)}{2}}$$

So, if we assume that n has at least one non-trivial Fermat witness, then by theorems 2 and 4, we arrive at the following conclusion:-

The percentage of Fermat witnesses in the set $\{1, 2, 3....n-1\}$ is least 50%.

4.4.3 Divisor Witnesses

For any natural composite number n, it is known that the number of divisor witnesses is least 2. Moreover, the number of divisors of n is of the order $O(\sqrt{n})$, more precisely, $\leq 2\sqrt{n}$, as mentioned in book **Introduction to Analytic Number Theory by Tom T. Apostol**, thus,

$$\boxed{2 \leq |D| \leq 2\sqrt{n} - 2}$$

4.4.4 Summary Table

Witness Type	**Lower Bound**	**Upper Bound**
GCD Witnesses	$n - \phi(n)$	$n - \phi(n)$
Fermat Witnesses(no non-trivial Fermat witness)	$n - \phi(n)$	$n - \phi(n)$
Fermat Witnesses($\exists$ non-trivial Fermat witness)	$n - \frac{\phi(n)}{2}$	$n - 1$
Divisor Witnesses	2	$2\sqrt{n} - 2$

Thus, we conclude that while the number of divisor witnesses and GCD witnesses is very small, the number of Fermat witnesses is typically very high, making them easy to find. As another typical example, consider $n = 415693$. There are only two divisor witnesses of n, namely 593 and 701. While there are more GCD witnesses, the proportion of GCD witnesses is still less than 1% (most numbers are relatively prime to n). By contrast, the proportion of Fermat witnesses is over 99%.

4.5 Fermat's Primality Test

We are given an integer n which we wish to classify as prime or composite. Repeat the following for k rounds:
1. Choose $a \in \{1, 2, 3, ...,$n-1$\}$ uniformly at random.
2. If $a^{n-1} \neq 1(mod n)$, return COMPOSITE and stop.
3. If we have not stopped after k rounds, then return prime.

Question: What is the error probability of this algorithm?

By Fermat's Little Theorem, we know that if n is prime, then $a^{n-1} = 1(mod\, n)$ holds for $\forall\ a \in \{1, 2, 3, ...,n-1\}$. Thus, error occurs when n is composite, but the algorithm returns it as prime. We need the probability of such

an error.

To find the probability, the general question arises:
Given that n is composite, how many Fermat witnesses does n have?

To address this problem, consider Theorem 4. If there is even just one non-trivial Fermat witness for n, then it follows that there are plenty of Fermat witnesses for n. Unfortunately, there exists a set of composite numbers for which there are zero non-trivial Fermat witnesses. These numbers are called **Carmichael numbers**.

4.5.1 Carmichael Numbers

A **Carmichael number** is a composite integer n such that

$$\forall a \in \{1, 2, 3, \ldots, n-1\}: \quad \text{if } \gcd(a, n) = 1, \text{ then } a^{n-1} \equiv 1 \pmod{n}.$$

Because this holds for all a coprime to n, the Carmichael numbers have **zero non-trivial Fermat witnesses**.

Upper Bound n	**Number of Carmichael Numbers** $\leq n$
100	0
1,000	1
10,000	7
100,000	43
1,000,000	255

4.5.2 Properties of Carmichael Numbers

- Carmichael numbers only have trivial Fermat witness, therefore, number of Fermat witnesses for Carmichael numbers is exactly $n - \phi(n)$. If we consider the fraction of Fermat witnesses for Carmichael numbers, that is, $\frac{n-\phi(n)}{n}$
  $= 1 - \frac{\phi(n)}{n}$
  By Euler's product formula [3], we get,

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

thus,

$$\text{Fraction of Fermat witnesses} = 1 - \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

which may be less than 50%.

- Carmichael numbers are odd, each having at least three distinct prime factors.
- They are square-free.
- They are infinite in number. [2]

From the perspective of primality testing, a Carmichael number, n, is likely to fail the Fermat Primality Test, because n has only trivial Fermat witnesses, and the number of trivial witnesses may be small compared to n, so it is unlikely that we'll find a Fermat witness to n's compositeness, even when the test is run for many iterations.

4.5.3 Summary

The Fermat Primality Test is a classic Monte Carlo algorithm, requiring k rounds, however, there are a set of integers (**Carmichael numbers**) for which it doesn't work well (produces high error probability, thus requires more iterations to produce better testing).

5 Miller Rabin Primality Testing

The Miller–Rabin Primality Test is a Monte Carlo algorithm, which, unlike the Fermat Primality Test, Miller–Rabin works correctly on every integer n. It always returns prime when n is actually prime. For any composite number — including Carmichael numbers — it returns composite with probability greater than $\frac{3}{4}$ in each round. As a result, the probability that a composite number passes all k rounds is less than 4^{-k}, making the error probability exponentially small. Thus, with enough rounds, Miller–Rabin provides a highly reliable probabilistic test for primality.

5.1 Idea

The Miller–Rabin test is based on using a new witness of compositeness. We've seen that finding a divisor witness proves n is composite. We've also seen that finding a Fermat witness proves that n is composite. A third way to prove that n is composite is to find a non-trivial square root of 1 mod n. The idea is based on the following theorem.

Theorem 5. *If p is prime, then all integer roots of $x^2 \equiv 1(mod\, n)$ satisfy $x = 1(mod\, n)$ or $x = -1(mod\, n)$.*

Proof. If $x^2 \equiv 1(mod\, n)$,
$\implies x^2 = 1 + qn$, for some natural number q,
$\implies x^2 - 1 = qn$
$\implies (x+1)(x-1) = qn$
Assuming $p > 2$, then p either divides $(x+1)$ or $(x-1)$.
If $p|(x-1) \implies x = 1(mod\, n)$
If $p|(x+1) \implies x = -1(mod\, n)$
Thus, all integer roots of $x^2 \equiv 1(mod\, n)$ satisfy $x \equiv 1(mod\, n)$ or $x \equiv 1(mod\, n)$.
□

If we consider the contrapositive of the above theorem, which says,

Given integers n and x, such that $x^2 \equiv 1(mod\, n)$. If $x \neq \pm 1(mod\, n)$, then n must be composite.

Note that, here x can be used as a witness to n's compositeness. Thus, we define a new type of witness.

5.2 Root Witness

Given a composite number n, we say that n is a root witness of n's compositeness, if $x^2 \equiv 1(mod\, n)$ and $x \neq \pm 1(mod\, n)$.

5.3 Logic behind the algorithm

We assume $n > 2$, and choose a randomly from $\{1, 2, 3, ...,$n-1$\}$. We also assume that n is odd, because if n is even, we immediately know that n is composite, since 2 is the only even prime number.

Given that n is odd, we consider $n - 1$, which must be even. Since $n - 1$ is even, it contains at least one factor of 2. Let's remove all the powers of 2 from $n - 1$. We are left with:

$$n - 1 = 2^r \cdot d, \quad \text{where } r > 0 \text{ and } d \text{ is odd.} \tag{1}$$

The Fermat primality test tells us that if

$$a^{n-1} \not\equiv 1 \pmod{n}, \tag{2}$$

then a is a Fermat witness of n.

On the other hand, if $a^{n-1} \equiv 1 \pmod{n}$, then we cannot conclude anything — n might still be prime.

We can rewrite the Fermat test condition using equation (1), as follows:

$$a^{2^r \cdot d} \not\equiv 1 \pmod{n}, \tag{3}$$

which implies that a is a Fermat witness to compositeness, so we return "composite" and halt.

Now suppose instead that:

$$a^{2^r \cdot d} \equiv 1 \pmod{n}. \tag{4}$$

Question: We haven't found a Fermat witness, but is there a test that we can do to look for a *root witness*?

Answer: We can rewrite the above equation as follows:

$$\left(a^{2^{r-1} \cdot d}\right)^2 \equiv 1 \pmod{n}. \tag{5}$$

Now we can ask whether (5) has a *root witness.*

If

$$a^{2^{r-1} \cdot d} \not\equiv \pm 1 \pmod{n}, \tag{6}$$

then we have found a *root witness* of compositeness, so we return "composite" and we're done.

Now suppose instead that:

$$a^{2^{r-1} \cdot d} \equiv \pm 1 \pmod{n}. \tag{7}$$

Question: Do we get another chance to try to find a root witness?

Answer: If

$$a^{2^{r-1}\cdot d} \equiv -1 \pmod{n}, \tag{8}$$

there's nothing more we can do. In this case we're done testing. Given that we haven't found any witness, we should return "prime" and stop.

However, if

$$a^{2^{r-1}\cdot d} \equiv 1 \pmod{n}, \tag{9}$$

and $r - 1 > 0$, then we do in fact get another chance to find a root witness of compositeness.

Question: Back in (8) we said that if $a^{2^{r-1}\cdot d} \equiv -1 \pmod{n}$, then we're done. How do we know that some lower exponent (some future square root) won't give us another opportunity to find a non-trivial square root of 1?

Answer: To witness a non-trivial square root of 1, we would need to again encounter an equation of the form

$$x^2 \equiv 1 \pmod{n},$$

where x is some lower power of a obtained by taking square roots. However, this can't happen. Observe that once we see that some power of a is congruent to 1 mod n, all future squarings will also be congruent to 1 mod n. So given that

$$a^{2^{r-1}\cdot d} \equiv -1 \pmod{n},$$

it is impossible for some later square root to be congruent to 1 mod n. Hence, no further root witness can be found.

5.4 Test

Since the algorithm (the next page) is a Monte Carlo algorithm, we will repeat the test for k iterations.

While the algorithm 1 is entirely correct, it is more computationally expensive than needed, because it requires first computing $a^{2^r \cdot d}$. This is achieved by starting with a^d and then repeatedly squaring that quantity r times. It is possible to restate the above algorithm where we compute only those powers of a that are needed. The following algorithm 2 shows the more efficient version.

Algorithm 1 Miller–Rabin Primality Test

Require: Integer $n > 2$, where n is odd; security parameter k
Ensure: Returns `COMPOSITE-Fermat`, `COMPOSITE-Root`, or `PROBABLY PRIME`

```
1: Express n − 1 = 2^r · d for some odd d
2: for i = 1 to k do
3:     Choose a ∈ {2, 3, ..., n − 2} uniformly at random
4:     Compute x = a^d mod n
5:     if x = 1 or x = n − 1 then
6:         continue to next iteration
7:     for j = 1 to r − 1 do
8:         x ← x^2 mod n
9:         if x = n − 1 then
10:            break and continue to next iteration
11:        else if x = 1 then
12:            return COMPOSITE-Root
13:    if x ≠ n − 1 then
14:        return COMPOSITE-Fermat
15: return PROBABLY PRIME
```

5.5 Analysis of Miller–Rabin Primality Test (Algorithm 2)

Why does Algorithm 2 only iterate up to $y = r-1$? Why not include $y = r$?

Suppose the algorithm has not terminated by the time it reaches $y = r-1$. Then it must be that

$$a^{2^{r-1} \cdot d} \not\equiv \pm 1 \mod n.$$

Now consider the case $a^{2^r \cdot d} \mod n$, which is equal to $a^{n-1} \mod n$.

- If $a^{n-1} \not\equiv 1 \mod n$, we have a **Fermat witness** and should return `COMPOSITE-Fermat`.
- If $a^{n-1} \equiv 1 \mod n$, but the previous value was not ± 1, we have a non-trivial square root of 1, i.e., a **root witness**, and should return `COMPOSITE-Root`.

In both cases, n is provably composite, and there is no need to check the value at $y = r$.

Algorithm 2 Miller–Rabin Primality Test: Optimised Version with k Iterations

Require: Integer $n > 2$, where n is odd; security parameter k
Ensure: Returns `COMPOSITE-Root`, `COMPOSITE-Fermat`, or `PROBABLY PRIME`

```
Express n − 1 = 2^r · d for some odd d
for i = 1 to k do
    Choose a ∈ {2, 3, ..., n − 2} uniformly at random
    Let y = 0
    Compute x = a^(2^y · d) mod n
    if x ≡ 1 (mod n) or x ≡ −1 (mod n) then
        continue to next iteration
    for y = 1 to r − 1 do
        Compute x = a^(2^y · d) mod n
        if x ≡ 1 (mod n) then
            return COMPOSITE-Root
        else if x ≡ −1 (mod n) then
            break and continue to next iteration
    return COMPOSITE_Fermat −
        return PROBABLY PRIME
```

Algorithm 2 performs k of the Miller–Rabin test. In practice, the test is repeated independently for k randomly chosen bases a. The algorithm halts early if any round returns `COMPOSITE`.

If n is prime, the test will always return `PROBABLY PRIME`.

For composite numbers, we will state the following theorem:

Theorem 6.

If n is composite, the probability that any individual round finds a witness of compositeness is greater than $\frac{3}{4}$.

Proof. We will omit the proof here, since it is non-trivial. The proof can be found here.[6]

□

Therefore, the probability that *none* of the k rounds detect compositeness is at most $\left(\frac{1}{4}\right)^k$, and the probability that *some* round detects it is

$$> 1 - \left(\frac{1}{4}\right)^k.$$

5.6 Time Complexity Analysis of Miller-Rabin Primality Test(Algorithm 2)

Given an odd integer $n > 2$ and a security parameter k, the Miller-Rabin primality test performs k rounds. Each round involves:

- Decomposing $n - 1 = 2^r \cdot d$: $\mathcal{O}(\log n)$
- Modular exponentiation $a^{2^y.d} \mod n$: $\mathcal{O}(\log n)$, for each y
- For $r - 1\, iterations\, of\, y = \mathcal{O}(\log n)$ squarings: $\mathcal{O}(\log n)$

Total time per round: $\mathcal{O}(\log^2 n)$
Total time over k rounds:

$$\boxed{\mathcal{O}(k \log^2 n)}$$

The overall time complexity of the Miller-Rabin primality test is:

$$\boxed{\mathcal{O}(k.m^2)}$$

where m is the number of bits of n

This is efficient even for large integers n, especially because k is typically a small constant (e.g., 20 or 40) in practical implementations.

5.7 Summary

The Fermat Primality Test can fail for certain composite numbers such as Carmichael numbers, which have very few Fermat witnesses. The Miller–Rabin test enhances the detection of compositeness by including tests for **root witnesses**, in addition to Fermat witnesses.

The Miller–Rabin test:

- Always returns the correct result when n is prime.
- Detects compositeness with high probability even when n is a Carmichael number.
- Time complexity = $O(k.m^2)$, where m is number of bits of n.

With each independent round, the reliability of the Miller–Rabin test increases exponentially.

More details of the Miller-Rabin Primality Testing algorithm can be found at [4], [5] and [7].

6 Experimental Results

6.1 Verification of number of different types of witnesses

We plotted graphs of number of divisor, GCD and Fermat witnesses for all natural numbers till 1000. The results were as follows:

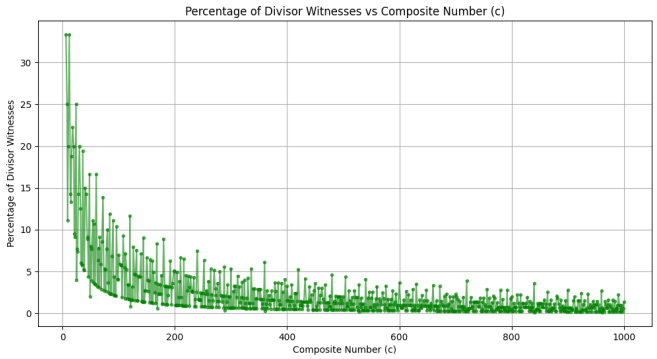


Figure 1: Plot of percentage of divisor-witnesses of n vs n

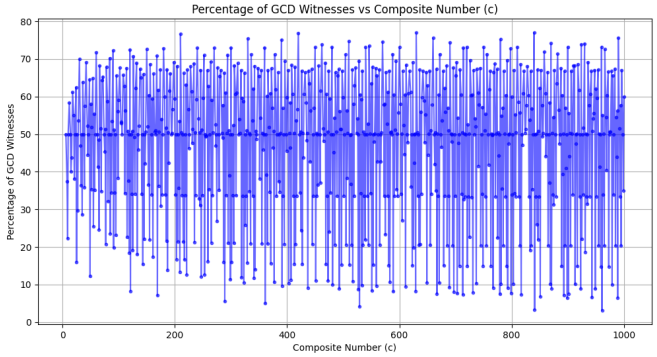


Figure 2: Plot of percentage of GCD witnesses of n vs n

From the following plot, we conclude the following:

- The percentage of divisor witness decreases as n decreases. As shown previously, the order of number of divisor witnesses is $O(\sqrt{n})$, thus the percentage of divisor witness roughly follow the rectangular hyberbola distribution, as shown in Figure 1.

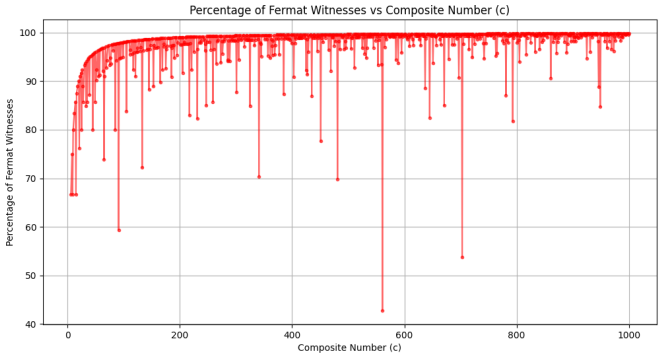


Figure 3: Plot of percentage of Fermat witnesses of n vs n

- The percentage of number of GCD witness seems to oscillate between the range $0 - 80\%$ in Figure 2. Thus, number of GCD witness is of order of n, thus, it is inefficient to find a GCD witness for n, even after k rounds.
- The percentage of Fermat witnesses seems to asymptotically converge to 100% as n increases, although, it has some outliers (**Carmichael numbers**) where, the percentage of Fermat witnesses come to as low as 35%. This was the reason of the failure of Fermat's primality testing algorithm.

6.2 Comparison between various primality testing algorithms

We compared different algorithms for primality testing and plotted their execution time for different primes upto 300 digits. Here is the desired plot:

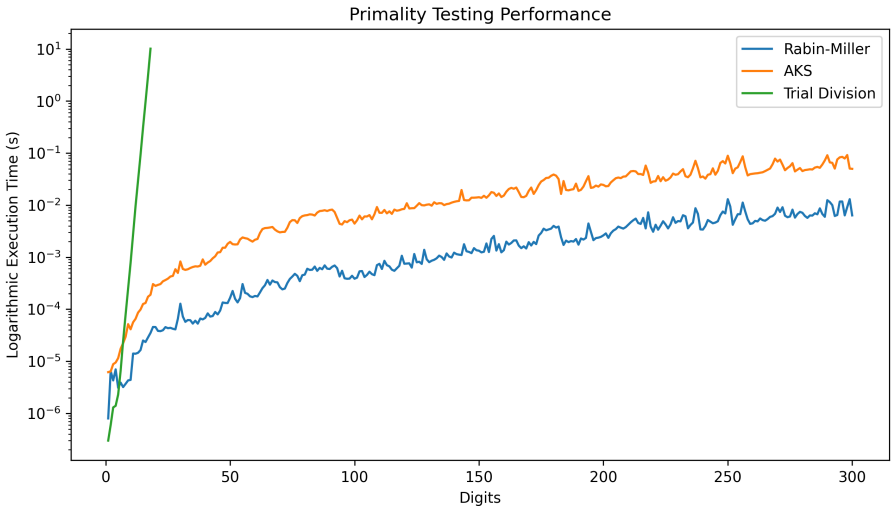


Figure: Execution time comparison for Miller-Rabin, AKS, and Trial Division primality tests across different number sizes (in digits). Logarithmic scale used on the y-axis for clearer visualization of time differences due to large disparity in execution times. Miller-Rabin, and AKS tests show moderate time increases with number size, while Trial Division shows a more dramatic time increase, becoming unfeasible after 18 digits.

Figure 4: Performance of various primality testing algorithms

As we can observe, the trial division shows a dramatic increases in time as n increases and becomes unfeasible after 18 digits. However, Miller-Rabin and AKS algorithms shows moderate time increases as size of input increases, with Miller-Rabin taking less execution time as compared to AKS algorithm.

6.3 Time taken to generate public-private Key pair in RSA using Miller Rabin algortithm

We test the performance of the Miller–Rabin primality test by measuring the time taken to generate a public–private key pair of a given bit length for RSA encryption. Here is the desired plot:

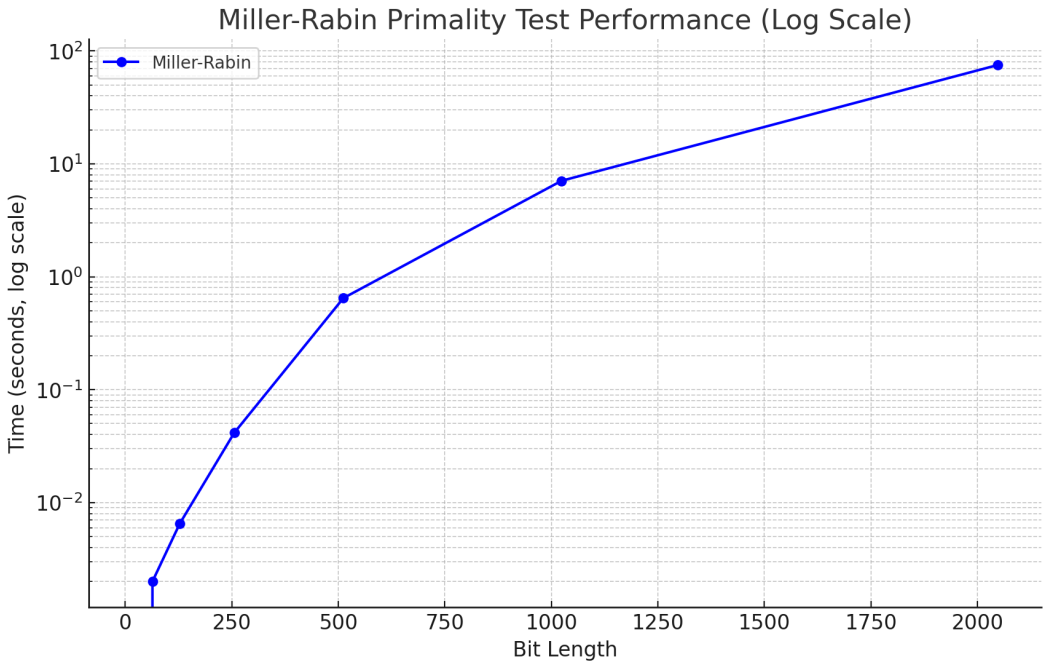


Figure 5: Performance of Miller Rabin Algorithm in generating public-private Key pair in RSA

The testing is performed on a local machine with the following specifications:

- **Processor:** 13th Gen Intel Core i9-13900HX CPU @ 2.20 GHz
- **RAM:** 16 GB
- **Operating System:** Windows 11
- **Python Version:** 3.11

7 An Application of Primality Testing(RSA Encryption)

To show the applications of primality testing in real life, we implemented a **multi-user RSA Encryption application**. This application securely encrypts and decrypts files for multiple users using RSA encryption. Each user has their own public and private key, which keeps their files safe. The application is designed using Object-Oriented Programming (OOP) to keep the code

organized and easy to expand in the future. This is the link to the GitHub repository containing our code.

7.1 How RSA works

RSA relies on the difficulty of factoring large integers. To generate the key pair, two large prime numbers, p and q, are selected. These primes should be chosen randomly and be significantly different from each other to enhance security.

Once p and q are chosen, their product $n = p \times q$ is computed. The values of p and q are kept secret, while n — which serves as the modulus for both the public and private keys — is made public.

The next step involves calculating Euler's totient function, $\phi(n) = (p-1)(q-1)$. After that, an integer e is selected as the public exponent such that it is relatively prime to $\phi(n)$. Finally, the private exponent d is calculated as the modular inverse of e modulo $\phi(n)$. This value d is used in the private key.

7.1.1 Encryption

When encrypting a message, the sender uses the public key (n, e) of the recipient to compute the ciphertext, where the ciphertext $= m^e (mod\, n)$. The m indicates the plaintext message.

7.1.2 Decryption

When decrypting an RSA encrypted message, the recipient uses their private key (n, d) to compute the plaintext message, where the plaintext message $= c^d (mod\, n)$.

7.2 Features

- **User Registration**: Allows users to register and generate their own RSA key pairs.
- **Key Management**: Users can download and store their private keys securely.
- **File Encryption**: Encrypts files using the recipient's public key.
- **File Decryption**: Requires the correct private key to decrypt a file.
- **GUI Interface**: Built with **Tkinter** for ease of use.

- **Miller-Rabin Primality Test**: Used to generate large prime numbers for key generation.

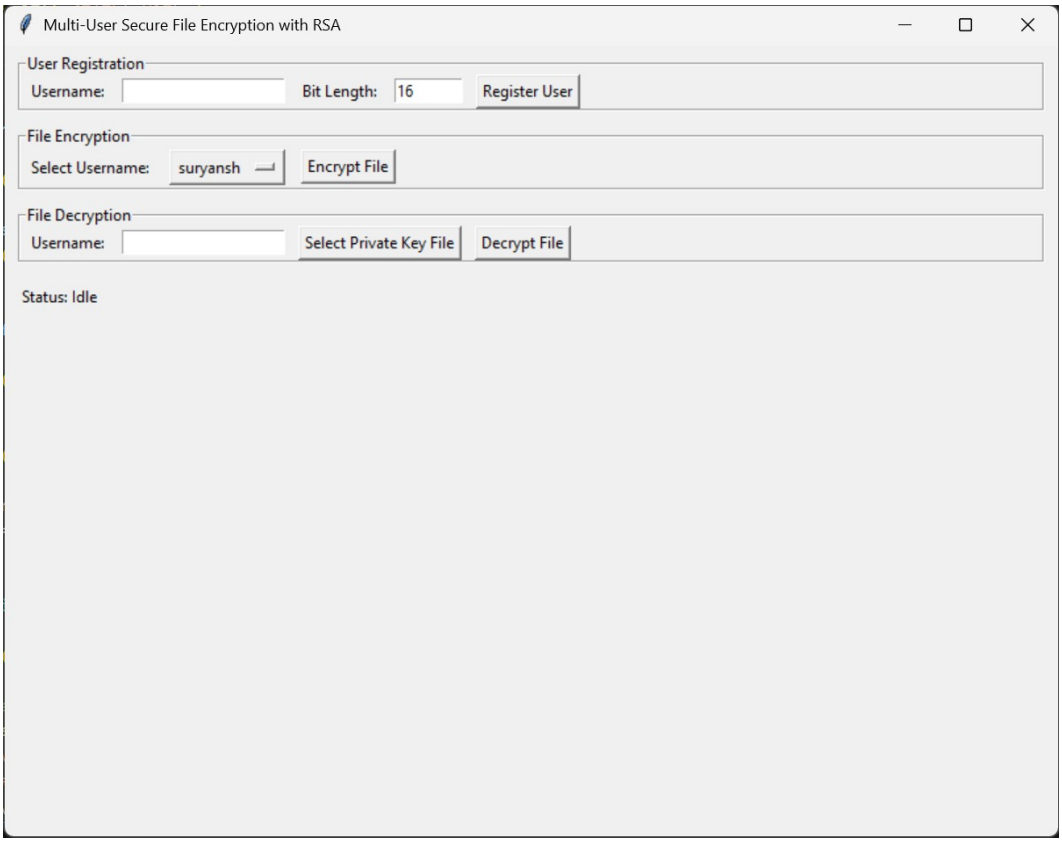


Figure 6: RSA Application Graphic User Interface

7.3 Security Considerations

- Private keys are stored securely and should never be shared.
- Encryption uses a secure RSA implementation with large prime numbers.
- The application prevents unauthorized decryption by enforcing correct private key verification.

7.4 Future Improvements

- Implement database storage for keys instead of local files.

- Add AES hybrid encryption for improved performance with large files.
- Introduce multi-factor authentication for user login.

8 Folder Structure and File Descriptions

The codebase implements an RSA-based file encryption and decryption system with a graphical user interface, along with modules for primality testing and performance benchmarking. Additional experiments explore topics such as Fermat's Little Theorem and GCD properties.

Experiments

- **divisor_witnesses.png**
  An image related to divisor witness tests.
- **fermat_little_theorem_test_on_composite_numbers.cpp**
  A C++ program testing Fermat's Little Theorem on composite numbers.
- **fermat_little_theorem_test_on_composite_numbers.py**
  A Python script testing Fermat's Little Theorem on composite numbers.
- **fermat_witnesses.png**
  Visualization of results for Fermat witness experiments.
- **gcd_witnesses.png**
  Visualization related to GCD testing or witness values.

RSA_application

- **gui.py**
  Tkinter GUI for single-user RSA encryption/decryption.
- **miller_rabin_performance.jpg**
  Image displaying performance results for the Miller-Rabin algorithm.
- **multiuser_gui.py**
  Multi-user Tkinter GUI supporting registration, key management, file encryption and decryption using RSA.
- **performance_results.md**
  Markdown file containing performance benchmarks and observations.

- **performance_testing.py**
  Script to benchmark the RSA and primality test implementations.

- **primality.py**
  Implements multiple primality tests: Miller-Rabin, square-root trial division, and an AKS-inspired test.

- **rsa.py**
  Contains core RSA logic: key generation, encryption/decryption, data conversion, and chunk processing.

time_comparisons

- **data**
  Contains prime number datasets used in performance testing.

- **src**
  Implements various primality testing algorithms:

  - `trial_division.py`: Trial division test.
  - `miller_rabin.py`: Miller-Rabin probabilistic test.
  - `aks.py`: AKS-style deterministic test.

- **main.py**
  Script to compare execution time of primality testing methods and generate performance plots.

- **primality_testing_performance.png**
  Plot showing time comparisons of the three primality algorithms.

Root-Level Files

- **.gitignore**
  Lists files and directories ignored by Git.

- **README.md**
  Project overview and setup instructions.

- **requirements.txt**
  Python dependencies required to run the project.

References

[1] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. Primes is in p. *Annals of Mathematics*, 160(2):781–793, 2004.

[2] W. R. Alford, Andrew Granville, and Carl Pomerance. There are infinitely many carmichael numbers. *Annals of Mathematics*, 139(3):703–722, 1994.

[3] T.M. Apostol. *Introduction to Analytic Number Theory*. Undergraduate Texts in Mathematics. Springer New York, 1998.

[4] Mor Harchol-Balter. Probability and computing, chapter 23: The miller-rabin test, n.d. Accessed: 2025-04-11.

[5] Michael O. Rabin. Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1):128–138, 1980.

[6] Rene Schoof. Four primality testing algorithms, 2008.

[7] Wikipedia contributors. Miller–rabin primality test — wikipedia, the free encyclopedia, 2025. Accessed: 2025-04-11.